COMP6258 Reproducibility Challenge: NegMerge: Sign-Consensual Weight Merging for Machine Unlearning

Kai Jie Liang [*], Adrian Low[†], Jingren Tan[‡], Abhishek Sengupta[§]

March 5, 2026

Paper Information:

Open Review forum page (ICML 2025): https://openreview.net/forum?id=ZbWXovStjD
The code is available at https://github.com/naver-ai/negmerge

Plan:

What is the motivation of the paper?

Machine unlearning aims to selectively remove specific training data from a trained model without the heavy computational cost of retraining from scratch. A common approach, Task Arithmetic, fine-tunes a model on the data to be forgotten to create a task vector, which is then subtracted from the original model. This requires generating many candidate models during validation and discarding all but one. This process is inefficient and wastes potentially useful information contained in the discarded candidate models.

What are the key claimed contributions of the paper?

NegMerge introduces a sign-consensual merging rule for machine unlearning. Rather than selecting a single best fine-tuned model from a validation pool, it computes task vectors from all candidates and builds a merged vector that keeps only the weight elements whose update sign is consistent across every candidate — sign-conflicting elements are masked to zero. This merged vector is then negated from the original model to drive forgetting.

The paper's central scientific claim is that unanimous sign agreement across candidates is a reliable signal that a weight element is genuinely tied to the forget set, while sign disagreement indicates noise from different training configurations or entanglement with general knowledge. This is what separates NegMerge from merging methods like TIES-Merging, which also handles sign conflicts but resolves them through voting for the purpose of general multi-task merging — NegMerge instead discards conflicting elements entirely, and its goal is specifically unlearning.

A direct consequence of the strict unanimity requirement is sparsity: the merged vector ends up modifying only 5–10% of weight elements, and the paper claims that greater sparsity correlates with stronger unlearning while keeping retain set performance stable.

On the practical side, the method reduces validation cost. Standard negation-based methods must tune a scaling coefficient λ across all n candidates, costing $\mathcal{O}(mn)$. NegMerge merges first and tunes λ on a single merged vector, bringing the cost down to $\mathcal{O}(m)$.

What experiments are you aiming to do?

Experiment 1: Validating the code

For the CLIP unlearning scenario, all runs use ViT-B/32 on two datasets: Cars and SVHN. ViT-B/32 is the lightest backbone and is the primary model used in all of the paper's deep-dive analyses. Cars represents a standard natural-image fine-grained recognition task while SVHN represents a severe domain shift as a digit recognition dataset, and together they cover the range of difficulty seen across all eight datasets in the original evaluation. 30 candidate models are trained by varying RandAugment configurations, with the number of

[*]kjl1a21@soton.ac.uk
[†]agxl1e22@soton.ac.uk
[‡]jt1r25@soton.ac.uk
[§]as8n25@soton.ac.uk

sequential augmentation transformations ranging from 1 to 3 and the transformation magnitude ranging from 1 to 10. Three baselines are used: Task Arithmetic (Single Best Model) as the direct predecessor, Uniform Merge to establish that simple averaging is insufficient, and TIES-Merging as the closest conceptual competitor since it also addresses sign conflicts but resolves them through voting rather than strict unanimity.

For the standard classifier unlearning scenario, all runs use ResNet-18 on CIFAR-10 with 10% random data forgetting. 27 candidate models are trained by varying training epochs (40, 50, or 60), weight decay (10^{-4}, 5×10^{-5}, 10^{-5}), and label smoothing (0, 0.05, 0.1). Four baselines are retained: Retrain as the gold standard, Fine-tuning, Task Arithmetic, and SalUn, which is the state-of-the-art competitor that uses gradient-based weight saliency to identify forget-relevant parameters, making it the most meaningful point of comparison for NegMerge. Four metrics are recorded: accuracy on the retain set (Acc_{D_r}), accuracy on the forget set (Acc_{D_f}), accuracy on the test set ($\text{Acc}_{D_{\text{test}}}$), and the Membership Inference Attack (MIA) score, which is the standard metric for verifying that data was genuinely unlearned from a privacy standpoint and not simply misclassified.

Experiment 2: Partial Sign-Consensus (k-of-n Agreement)

The absolute core of NegMerge is the strict unanimity rule, which the authors describe as an AND gate. Rather than using a voting mechanism as in TIES-Merging, NegMerge requires 100% consensus across all candidates, claiming this isolates the exact forget-set parameters. However, it remains unclear whether 100% consensus is the optimal threshold or simply a convenient heuristic.

This experiment sweeps the consensus threshold, for example retaining weight elements where 80%, 90%, or 100% of candidates agree, in order to directly map the relationship between sparsity, unlearning performance, and the level of consensus required. If 80% agreement produces better retain accuracy while still achieving effective forgetting, it would indicate that the strict unanimity rule is too aggressive and over-prunes the network. If 100% consensus remains the Pareto-optimal frontier across all thresholds, it would mathematically validate the paper's central hypothesis.

Experiment 3: Controlled Experiment on Candidate-Pool Diversity and Pool Quality

NegMerge assumes that updates caused by the forget set will produce stable weight directions across different training runs, while noise introduced by different training configurations will fluctuate and therefore be removed during merging. This means that the effectiveness of the method is strongly dependent on the composition of the candidate pool. The paper briefly notes that poorly constructed task vectors can introduce noise and recommends using reasonable hyperparameters, but it does not systematically study how sensitive the method is to the quality and diversity of the candidate models. Because NegMerge requires strict unanimous sign agreement across all candidates, the merged vector may be highly sensitive to the structure of the pool.

This experiment evaluates two boundary conditions that reflect realistic automated validation pipelines. First, we restrict hyperparameter diversity by fixing the learning rate, weight decay, and augmentation settings, while varying only the random seed across candidate models. This tests whether stochasticity from different training seeds alone is sufficient for NegMerge to identify stable updates. If retain accuracy degrades under this condition, it would indicate that the method depends on hyperparameter variation rather than pure training randomness to correctly remove noisy updates.

Second, we introduce poorly trained candidates into the validation pool by using unreasonable hyperparameter settings, such as excessively high learning rates or heavy weight decay. Because NegMerge requires unanimous sign agreement across all candidates, a single unstable model could prevent many weight updates from being retained in the merged vector. If the presence of such models causes the merged task vector to collapse toward zero, it would suggest that NegMerge cannot simply use all candidate models produced during validation and instead requires manual filtering of poor runs.

Together, these tests examine whether the method is genuinely robust to different candidate pools or whether it relies on a specific balance of candidate diversity and training quality to function properly.

Experiment 4: Layer-Wise Consensus Thresholds Guided by the Paper's Depth Observation

In Table 6 of the paper, the authors observe that shallow layers have lower consensus and higher sparsity because general knowledge encoded in those layers is more easily distorted during fine-tuning. In contrast, deeper layers encode stable, task-specific knowledge and show higher consensus across candidates.

This experiment tests whether the paper's own internal logic holds by applying a layer-wise strategy: forcing the NegMerge task vector to zero for shallow layers, effectively leaving them unchanged, and applying NegMerge only to the deeper layers. If unlearning performance holds or improves under this configuration, it would confirm the authors' hypothesis about layer depth and simultaneously make the method more computationally efficient by reducing the number of parameters it needs to modify.

Experiment 5: Backdoor-Trigger Unlearning as a Hard Forget Set

The standard evaluation in NegMerge uses random 10% data forgetting, which is a diffuse target where the model encodes the forgotten features across many parameters. A backdoor trigger, such as a localized pixel patch, represents a concentrated, specific, and unnatural parameter perturbation, making it a much harder and more demanding test case.

If NegMerge's sign-consensus genuinely acts as a precise scalpel that isolates only the supervision of the forget set, it should be able to surgically remove the backdoor trigger while leaving clean accuracy completely untouched. If the strict unanimity rule fails to identify the backdoor-relevant weights and therefore fails to unlearn it, this would expose a critical limitation: unanimous consensus may be too strict to capture concentrated and adversarially inserted behaviors.